

PocketLab BLE GATT and UUID Notes

A compact reference for custom BLE services, characteristics, properties, and addressing.

BLE Device
GATT Server
Service
Characteristic
Descriptor

1. UUIDs

A UUID is the address used to identify a BLE service, characteristic, or descriptor. The client uses the service UUID to find a subsystem and the characteristic UUID to find a specific value or command inside it.

2. Standard vs. custom UUIDs

Bluetooth SIG defines standard short UUIDs, such as 0x180F for Battery Service. Private projects should use custom 128-bit UUIDs because commands such as SET_FREQ or OUTPUT ON are application-specific.

3. GATT server and client

GATT means Generic Attribute Profile. In PocketLab, the ESP32 is the GATT server because it owns the data structure. The phone or laptop is the GATT client because it connects, discovers, reads, writes, and subscribes.

4. Services

A service groups related characteristics. One PocketLab service may contain command, response, device-state, and firmware-information characteristics. Larger systems can later separate instruments into multiple services.

5. Characteristics

A characteristic holds a value and declares which operations are permitted. Its UUID identifies it; its properties define whether it can be read, written, notified, or indicated.

6. Characteristic properties

Read: client asks for the current value. **Write with response:** client sends a value and waits for a GATT acknowledgment; nRF Connect calls this a Request. **Write without response:** client sends without acknowledgment; nRF Connect calls this a Command. **Notify:** server pushes updates to a subscribed client. **Indicate:** like notify, but the client confirms receipt.

7. Descriptors

Descriptors provide metadata about a characteristic. The most common is 0x2902, the Client Characteristic Configuration Descriptor, which enables notifications or indications. BLE libraries usually manage it automatically when the app subscribes.

8. PocketLab current structure

The minimal test uses one custom service and one characteristic with Read, Write, and Write Without Response. The phone writes PING, the ESP32 onWrite callback runs, and a later read returns PING.

9. Recommended PocketLab structure

Use a Command RX characteristic for phone-to-ESP32 writes and a Response TX characteristic for ESP32-to-phone reads or notifications. A separate Device State characteristic can expose current settings and notify when they change.

10. RX and TX naming

RX and TX are named from the ESP32 peripheral perspective. Command RX means the ESP32 receives commands there. Response TX means the ESP32 transmits responses there.

11. Advertising vs. GATT

Advertising is how the ESP32 announces itself before connection, using a small packet containing items such as device name or service UUID. GATT is the full data structure discovered after connection. The advertisement does not need to list every characteristic.

12. How a BLE write is addressed

A write needs three things: the active device connection, the service UUID, and the characteristic UUID. All three must belong to the same current connection. This is why stale Device objects can cause confusing failures even when the UUIDs are correct.

13. UUIDs do not define behavior

A UUID is only an identifier. Behavior comes from characteristic properties, server callbacks, and the application protocol. Both sides must agree that a given characteristic carries commands such as PING or SET_FREQ 1000.

14. Useful analogy

Think of a BLE service as an API namespace, a characteristic as an endpoint, a UUID as the endpoint address, Read as GET, Write as POST or PUT, and Notify as a server-pushed event. The analogy is not exact, but it is useful.

Custom UUID Rules

Use these rules when creating PocketLab services and characteristics.

Format	Use 128-bit UUIDs: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
Characters	Hexadecimal only: 0-9 and a-f. Uppercase or lowercase is acceptable.
Generation	Generate a random UUID with PowerShell: [guid]::NewGuid()
Short UUIDs	Do not invent custom 16-bit or 32-bit UUIDs; those spaces are intended for Bluetooth SIG assignments.
Uniqueness	Keep every service and characteristic UUID unique within the design.
Avoid	Do not use 00000000-0000-0000-0000-000000000000.
Behavior	The UUID does not determine Read, Write, Notify, or protocol meaning. Code and properties do.

Example PocketLab UUID family

```
Service: 7a110001-2d44-4a73-a8f1-3d9ea01c0001
Command RX: 7a110002-2d44-4a73-a8f1-3d9ea01c0001
Response TX: 7a110003-2d44-4a73-a8f1-3d9ea01c0001
Device State: 7a110004-2d44-4a73-a8f1-3d9ea01c0001
```

Recommended PocketLab GATT layout

Object	Properties	Purpose
PocketLab Service	-	Groups the custom PocketLab protocol.
Command RX	Write, Write Without Response	Phone sends commands to the ESP32.
Response TX	Read, Notify	ESP32 returns command responses.
Device State	Read, Notify	Current frequency, waveform, amplitude, offset, and output state.

PocketLab communication flow

```
ESP32 advertises PocketLab service
-> Phone discovers and connects
-> Phone discovers services and characteristics
-> Phone writes a command to Command RX
-> ESP32 parses the command in onWrite()
-> ESP32 sends a response through Response TX
-> Phone receives the response by read or notification
```

Key rule

After connecting and discovering services, keep one authoritative connected Device object and use that same object for reads, writes, notifications, and connection checks.